



Universidade de Brasília

Departamento de Ciência da Computação

Recursividade



Exemplo: soma_recursiva

- Suponha que tenhamos o seguinte código, que calcula a soma $n + (n-1) + \dots + 2 + 1$ de forma recursiva:

```
int soma_rec (int n)
{
    if (n < 1)
        return 0;
    else
        return n + soma_rec(n-1)
}
```

- Vamos gerar o código correspondente em assembly RISC-V.



Exemplo: soma_recursiva

- ♦ O parâmetro n corresponde ao registrador **a0**.
- ♦ Devemos inicialmente colocar um rótulo para a função, e salvar o endereço de retorno **ra** e o parâmetro **a0**:

Soma_rec:

addi sp, sp, -8	# prepara a pilha para receber 2 itens
sw ra, 4(sp)	# empilha ra (End. Retorno)
sw a0, 0(sp)	# empilha a0 (n)

- ♦ Na primeira vez que soma_recursiva é chamada, o valor de **ra** que é armazenado corresponde ao endereço que está na rotina chamadora.



Exemplo: soma_recursiva

- ◆ Vamos agora compilar o corpo da função. Inicialmente, testamos se $n < 1$:

```
slti t0, a0, 1  
beq t0, zero, L1      # se  $n \geq 1$ , vá para L1
```

- ◆ Se $n < 1$, a função deve retornar o valor 0. Não podemos nos esquecer de restaurar a pilha.

```
add a0, zero, zero      # valor de retorno é 0  
addi sp, sp, 8          # remove 2 itens da pilha  
ret                     # retorno
```

- ◆ *Por que não carregamos os valores de a0 e ra antes de ajustar sp??*



Exemplo: soma_rekursiva

- Se $n \geq 1$, decrementamos n e chamamos novamente a função soma_rekursiva com o novo valor de n .

```
L1:    addi a0, a0, -1    # argumento passa a ser (n-1)
        call soma_rec    # calcula a soma para (n-1)
```

- Quando a soma para $(n-1)$ é calculada, o programa volta a executar na próxima instrução. Restauramos o endereço de retorno e o argumento anteriores, e incrementamos o apontador de topo de pilha:

```
lw t0, 0(sp)    # restaura o valor de n
lw ra, 4(sp)    # restaura o endereço de retorno
addi sp, sp, 8   # retira 2 itens da pilha.
```



Exemplo: soma_recursiva

- ◆ Agora o registrador **a0** recebe a soma do argumento antigo **t0** com o valor atual em **a0** (soma_recursiva para $n-1$):

```
add a0, a0, t0          # retorne n + soma_rec(n-1)
```

- ◆ Por último, voltamos para a instrução seguinte à que chamou o procedimento:

```
ret                    # retorne para a chamadora
```



Listagem

Soma_rec:

```
    addi sp, sp, -8      # prepara a pilha para receber 2 itens
    sw ra, 4(sp)         # empilha $ra (End. Retorno)
    sw a0, 0(sp)         # empilha $a0 (n)
    slti t0, a0, 1       # testa se n < 1
    beq t0, zero, L1     # se n >= 1, vá para L1
    add a0, zero, zero   # valor de retorno é 0
    addi sp, sp, 8       # remove 2 itens da pilha
    ret                 # retorne para depois de jal
```

L1:

```
    addi a0, a0, -1      # argumento passa a ser (n-1)
    call soma_rec        # calcula a soma para (n-1)
    lw t0, 0(sp)         # restaura o valor de n
    lw ra, 4(sp)         # restaura o endereço de retorno
    addi sp, sp, 8       # retira 2 itens da pilha.
    add a0, a0, t0       # retorne n + soma_recursiva(n-1)
    ret                 # retorne para a chamadora
```



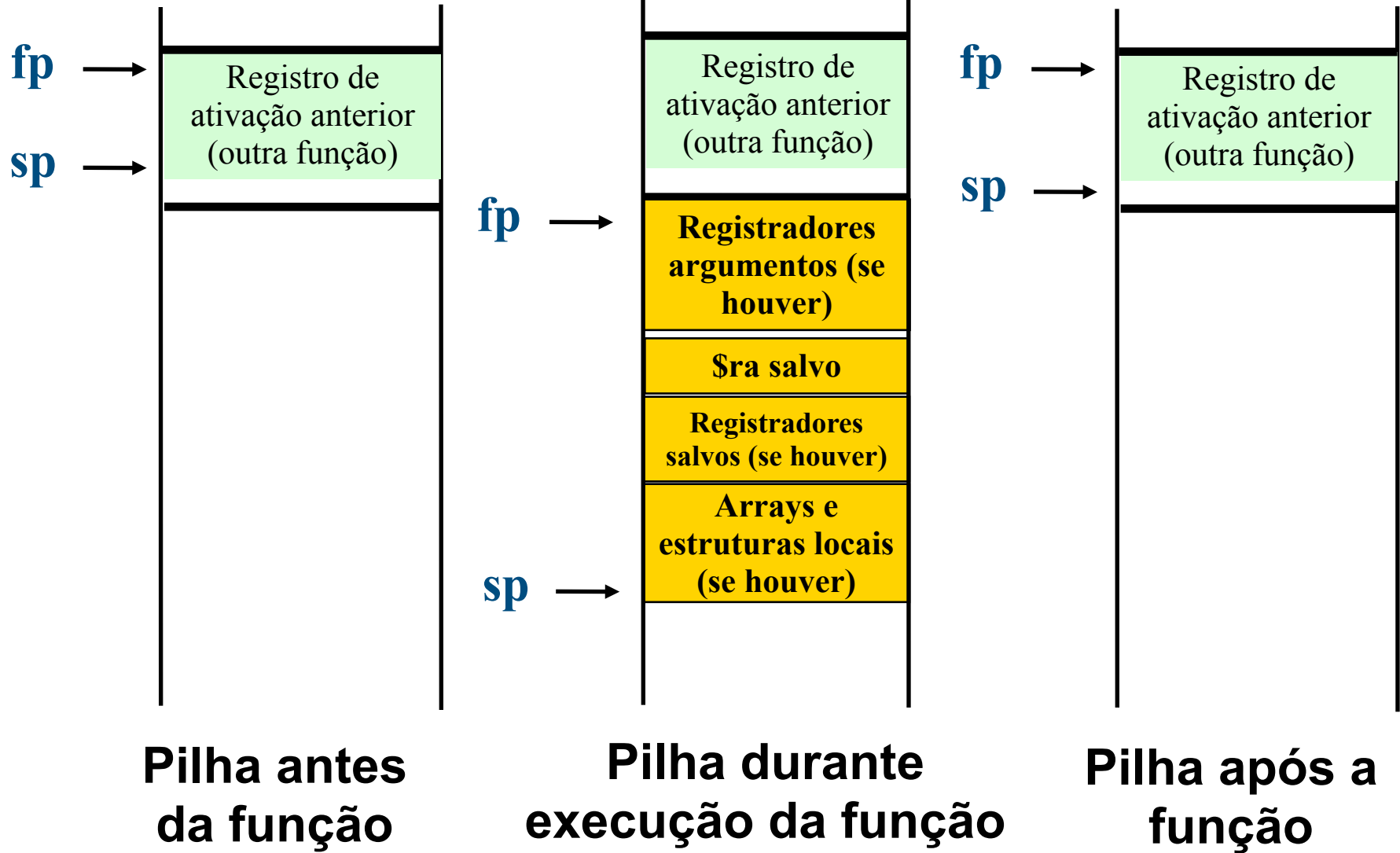
O que “deve” ser preservado?

Preservado	Não Preservado
Registradores s0-s11	Registradores t0-t6
Stack Pointer sp	Registradores a0-a7
Pilha acima do sp	Pilha abaixo do sp
Registrador de retorno ra	
Frame Pointer (fp) Global Pointer (gp) se utilizados	



Alocando espaço para novos dados locais na pilha

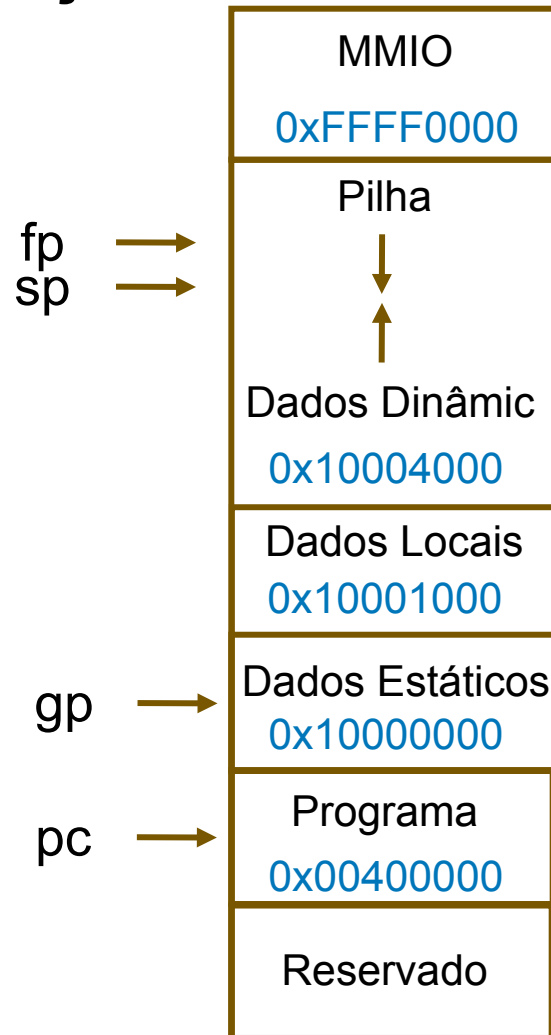
- Frame de Procedimento (Registro de Ativação)
 - Armazenar variáveis locais a um procedimento
 - Facilita o acesso a essas variáveis locais ter um apontador estável *fp*





Ex.: Alocação de memória (SPIM e RARS)

Endereço



Espaço para procedimentos armazenarem informações

Heap: Espaço explicitamente criado
malloc: apontadores em C

Variáveis estáticas declaradas uma vez para todo programa

Programa Código de Máquina

Exercício:

- Implemente uma rotina que calcule o enésimo valor da Série de Fibonacci
 $\{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots\}$

Fibonacci: # a0=n v0=Fibonacci(n)

■ ■ ■ ■

onde

- $\text{Fibonacci}(0) = 0$,
- $\text{Fibonacci}(1) = 1$
- $\text{Fibonacci}(2) = 1$ e
- $\text{Fibonacci}(n) = \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$

Uma solução recursiva e uma não-recursiva.
Qual possui maior desempenho?

